

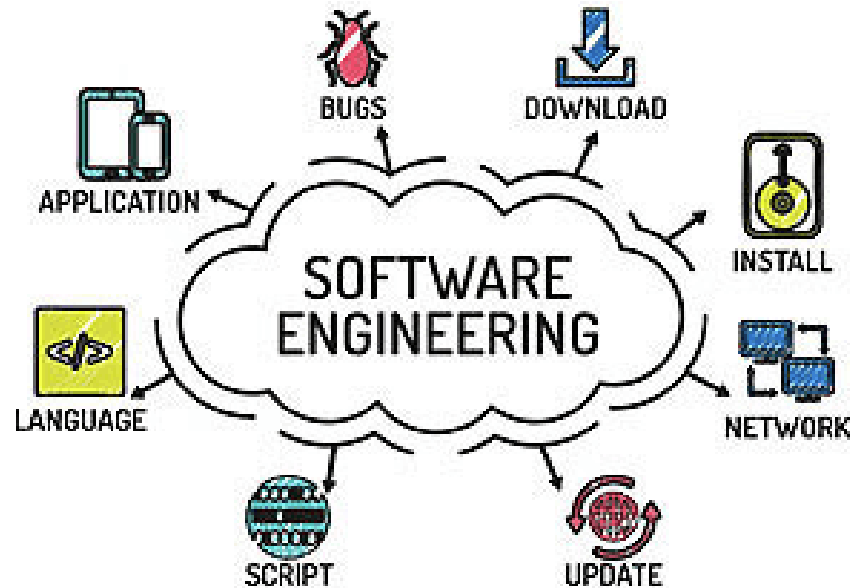


UIT
TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ THÔNG TIN

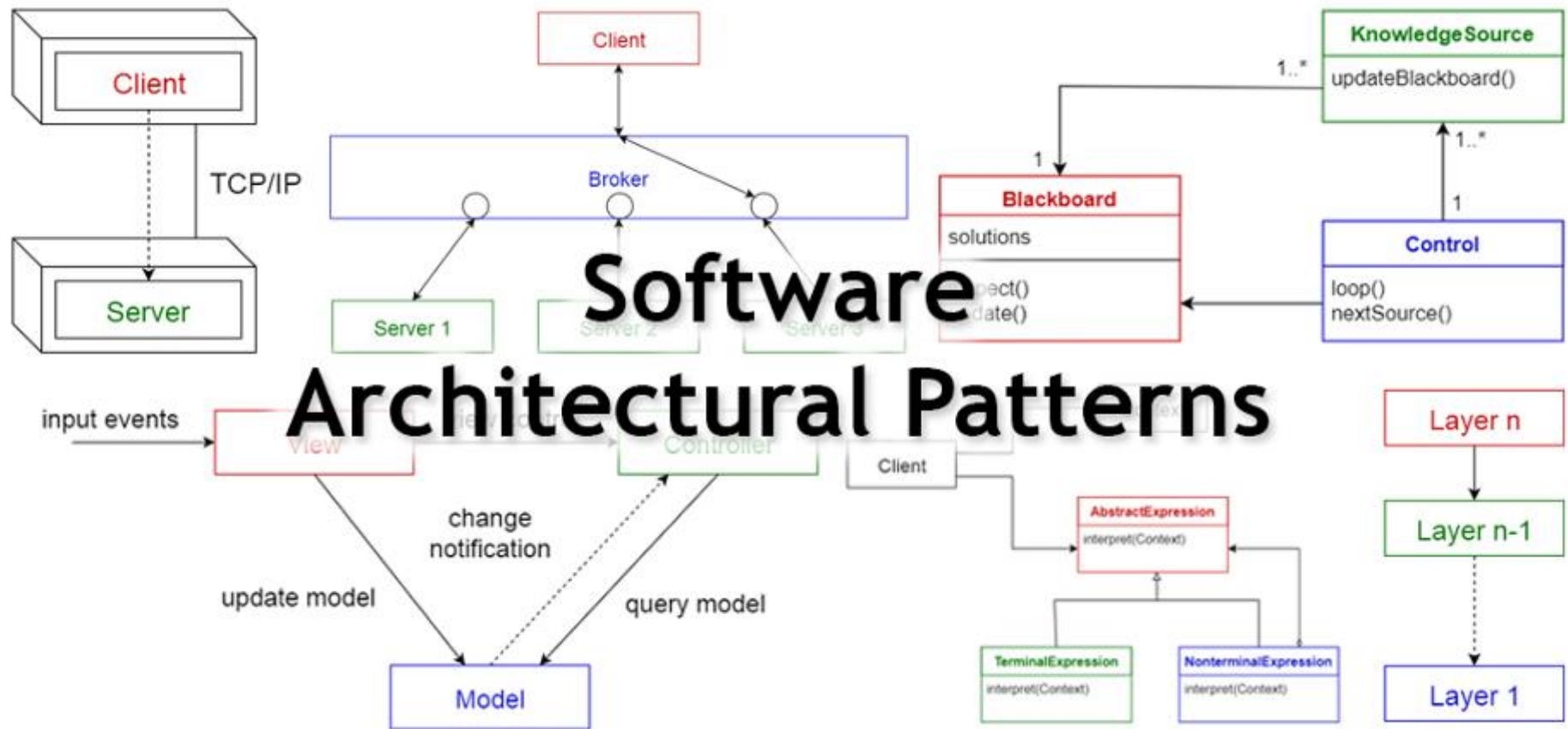


NHẬP MÔN CÔNG NGHỆ PHẦN MỀM

Introduction To Software Engineering



Kiến Trúc Phần Mềm



NỘI DUNG MÔN HỌC

- Tổng quan về Công nghệ phần mềm
- Xác định và mô hình hóa yêu cầu phần mềm
- **Thiết kế phần mềm**
- Cài đặt phần mềm
- Kiểm thử và bảo trì
- Đồ án môn học

THIẾT KẾ PHẦN MỀM

- **Thiết kế hệ thống**
- **Thiết kế đối tượng**
- **Thiết kế dữ liệu**
- **Thiết kế giao diện**

I. Thiết kế hệ thống

- Kiến trúc hệ thống
- Một số kiến trúc hệ thống

I.1 Kiến trúc hệ thống

Khái niệm:

Hệ thống được cấu tạo bởi các thành phần nào và mối liên hệ giữa các thành phần đó.

I.1 Kiến trúc hệ thống (tt)

Tầm quan trọng của kiến trúc:

- Ảnh hưởng hiệu quả hoạt động và an toàn hệ thống:
 - Tốc độ xử lý
 - Tính chịu lỗi
 - Tính bảo mật
- Ảnh hưởng chi phí triển khai, vận hành và bảo trì hệ thống.
- Ảnh hưởng trực tiếp đến giai đoạn thiết kế.

I.1 Kiến trúc hệ thống (tt)

Các bước thiết kế kiến trúc:

- Phân rã hệ thống -> xác định các thành phần;
- Bố trí các thành phần;
- Thiết lập mối quan hệ giữa các thành phần.

I.2 Một số kiến trúc hệ thống

■ Kiến trúc đơn lập

■ Kiến trúc phân tán:

- Kiến trúc 2 lớp
- Kiến trúc 3 lớp
- Kiến trúc đa tầng
- Kiến trúc Peer-To-Peer

I.2.1 Kiến trúc đơn lập

- **Đặc điểm:**
 - Là một thể thống nhất.
 - Các thành phần tự do tương tác.
- **Ưu điểm:**
 - Dễ lập trình và triển khai.
 - Tốc độ xử lý.
- **Khuyết điểm:**
 - Khó bảo trì, nâng cấp.
 - Không chia sẻ dữ liệu.

I.2.2 Kiến trúc phân tán

- Kiến trúc 2 lớp (Client – Server)
- Kiến trúc 3 lớp
- Kiến trúc đa tầng
- Kiến trúc Peer-To-Peer (P2P)

I.2.2.1 Kiến trúc Client – Server

- **Đặc điểm:**
 - Phân làm hai phân hệ Client và Server.
 - Server cung cấp dịch vụ, Client sử dụng dịch vụ.
 - Client gửi các yêu cầu đến Server.
- **Ưu điểm:**
 - Chia sẻ dữ liệu và đồng bộ.
 - Dễ bảo trì, nâng cấp.
- **Khuyết điểm:**
 - Tốc độ xử lý.
 - Chi phí triển khai.

I.2.2.1 Kiến trúc Client – Server (tt)

- **Thick Server (Thin-Client):**
 - Server = Dữ liệu + **Xử lý**
 - Client = Giao diện
- **Thick Client (Thin-Server):**
 - Server = Dữ liệu
 - Client = Giao diện + **Xử lý**

Ví dụ Kiến trúc Client-Server

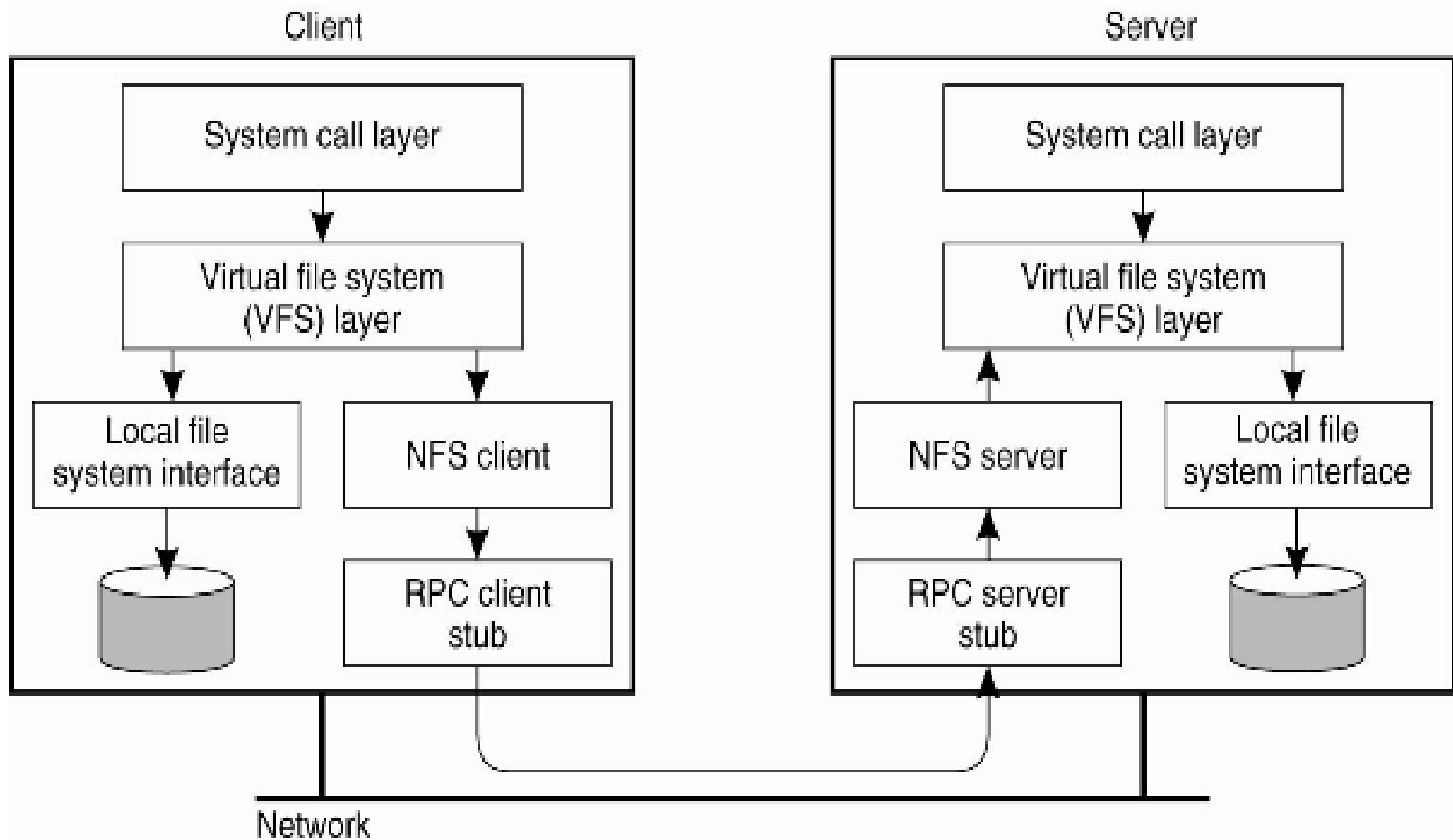
1) SQL Server:

- Server chứa CSDL thật sự, thực hiện việc đánh chỉ mục dữ liệu, viết các SP, tạo table...
- Client là Management Studio, cho phép connect đến các CSDL khác nhau.

Đây là Thick-Server.

2) File Server (ví dụ ftp server):

Ví dụ Kiến trúc Client-Server (tt)



Ví dụ Kiến trúc Client-Server (tt)

- Kiến trúc này có 2 lớp chính là Client và Server **kết nối với nhau thông qua giao thức RPC** (Remote Procedure Call - giao thức gọi hàm từ xa).
- **VFS Layer** (lớp quản lý file ảo) thực chất là **một thư viện được tích hợp vào**, *không phải là một tier* vì việc trao đổi dữ liệu với lớp này được thực hiện trên bộ nhớ của cùng 1 ứng dụng chứ không phải truyền từ ứng dụng này sang ứng dụng khác.
- **Các tier là những ứng dụng chạy độc lập và trao đổi với nhau theo các giao thức riêng được định nghĩa sẵn.**
-> Ví dụ ở đây là giao thức RPC, một số giao thức khác thường gặp như HTML, RMI, ...

I.2.2.2 Kiến trúc 3 lớp

- **Đặc điểm:** phân làm 3 phân hệ
 - **Presentation layer:** Giao diện người dùng.
 - **Business layer:**
 - + Thư viện xử lý
 - + Application server
 - **Data layer:**
 - + Dịch vụ dữ liệu
 - + Database server

Tương tác theo quy tắc “Thang máy”: **không tương tác vượt tầng.**

- **Ưu khuyết điểm:** tương tự kiến trúc Client-Server.

1.2.2.2 Kiến trúc 3 lớp (tt)

- ❖ **Thư viện xử lý:** chạy chung vùng nhớ của ứng dụng sử dụng thư viện.
 - Thư viện liên kết tĩnh (.lib): đưa code vào file .exe -> kích thước file .exe lớn.
 - Thư viện liên kết động (.dll): không cần biên dịch lại nhưng phải update file .dll mới.

- ❖ **Application Server:**

Là một dạng thư viện mới, không cần người lập trình phải update khi có thay đổi trên thư viện này.

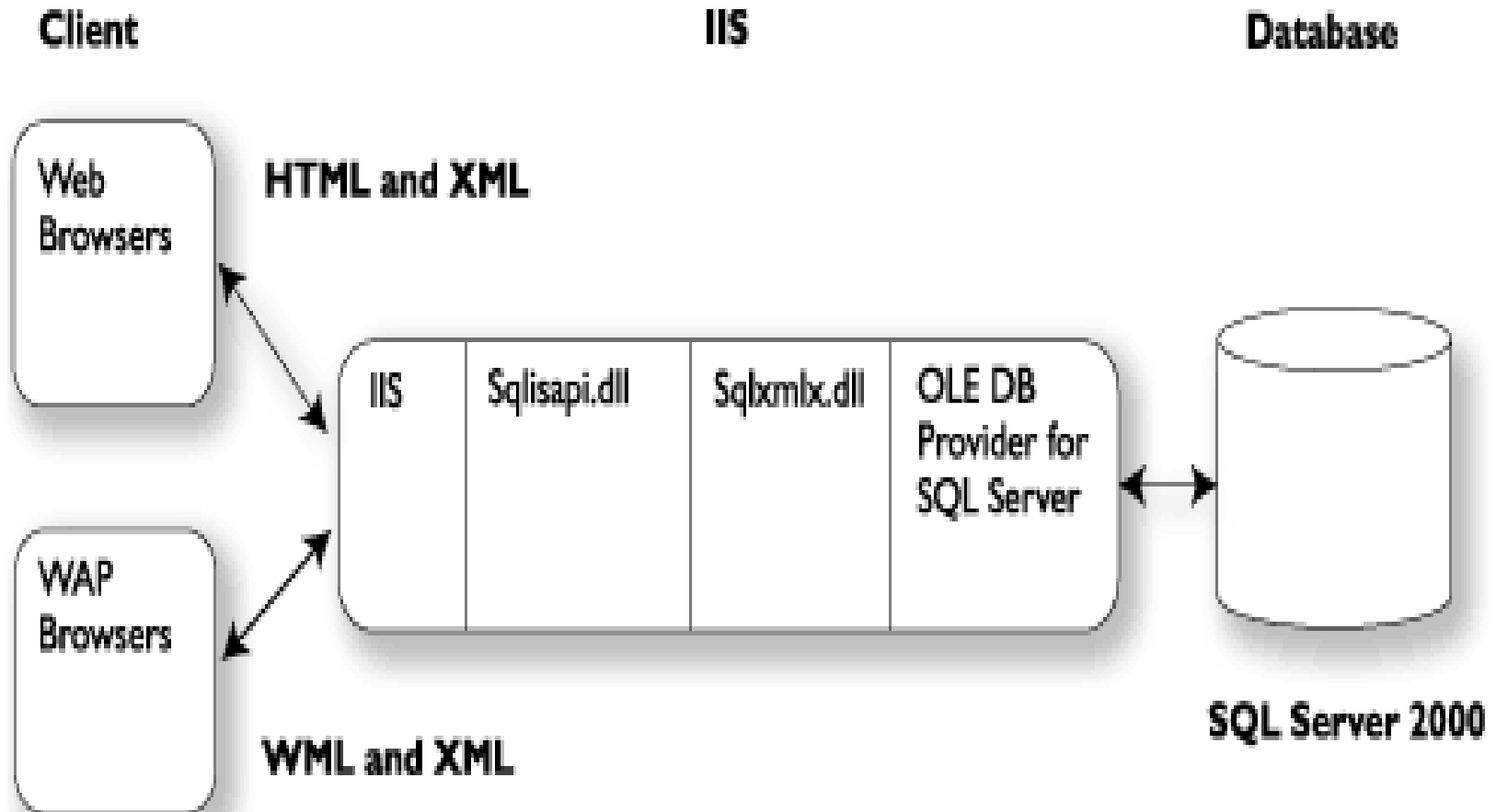
Không chạy chung vùng nhớ của ứng dụng, trả về các đối tượng đã được tạo ở một nơi khác -> Giao thức tạo đối tượng từ xa: RMI, COM, CORBA.

Ví dụ Kiến trúc 3 lớp

Các ứng dụng web thường theo kiến trúc 3 lớp:

- **Presentation layer (Client):** trình duyệt.
- **Business layer (Web server):** ví dụ IIS (Internet Information Server) của Microsoft.
 - + Chứa các trang .html; các file script thực hiện một số chức năng nào đó (.asp/.aspx) của ứng dụng web.
 - + Đóng vai trò như một middleware.
 - + Sử dụng các chức năng của Thư viện xử lý/Application server.
- **Data layer (Database server):** ví dụ SQL Server.

Ví dụ Kiến trúc 3 lớp (tt)



Ví dụ Kiến trúc 3 lớp (tt)

- + **Browser** chạy với 2 giao thức **WML/HTML** kết nối với web sever.
- + **Web server** là IIS sẽ phục vụ yêu cầu của Browser thông qua giao thức WML hoặc HTML tùy theo trình duyệt.
- + **Web Server** lấy dữ liệu từ SQL Server thông qua **các thư viện sqlisapi.dll và sqlxmlx.dll** theo giao thức riêng được xác lập nhờ vào trình điều khiển (driver) OLE DB Provider for SQL Server.

Triển khai ứng dụng Web

C1: Mua dịch vụ Hosting: Đăng ký một tên miền để chạy ứng dụng ở Web Browser (truy cập lên host).

C2: Demo trên máy cá nhân khi báo cáo đồ án.

C3: Nối mạng LAN:

- Một máy làm Client, chứa Browser
- Một máy làm Server, chứa IIS và SQL Server

Lưu ý: không nên xài host free vì:

- Có thể bị nhiễm virus từ server.
- Không ổn định: có thể làm hư file script...

I.2.2.3 Kiến trúc đa tầng

- **Đặc điểm:**
 - Mở rộng kiến trúc 3 lớp.
 - Phân làm nhiều tầng xử lý.
- **Ưu khuyết điểm:** tương tự kiến trúc Client – Server.

I.2.2.4 Kiến trúc Peer-to-Peer

- **Đặc điểm:**
 - Là kiến trúc phân tán, triển khai trên nhiều máy (nút).
 - Các nút tương tác được với nhau, mỗi nút đóng vai Client – Server.
 - Chia sẻ dữ liệu và xử lý.
- **Ưu điểm:**
 - Dễ triển khai, không cần server trung tâm.
 - Không gian lưu trữ và khả năng xử lý dàn trải.
- **Khuyết điểm:**
 - Khó lập trình và quản lý dữ liệu.

1.2.2.4 Kiến trúc Peer-to-Peer (tt)

Kiến trúc này được phát triển thành:

- Grid trong mạng nội bộ;
- Cloud trên Internet.

So sánh với Kiến trúc Cluster:

- **Kiến trúc Peer-to-Peer:** hệ thống bao gồm nhiều máy khác nhau, các máy trong hệ thống có thể khác hệ điều hành.
- **Kiến trúc Cluster:** dùng cab để *kết nối main board* chứa CPU, RAM, HDD, ... của các máy lại nhằm tận dụng tài nguyên của các máy (để nâng cao tốc độ xử lý...).

MVC

- MVC không phải là kiến trúc hệ thống mà là ***mô hình cài đặt các xử lý của ứng dụng.***
- Mỗi một xử lý sẽ có một MVC riêng cho nó, vì vậy mỗi một Form nên thực hiện một xử lý chính thôi (***tìm kiếm/chỉnh sửa/thêm mới***).
 - Model: tương tác với dữ liệu, gọi các stored procedure...
 - View: tương tác với người dùng (nhập/xuất thông tin...).
 - Controller: cài đặt các hàm xử lý.

MVC (tt)

Ưu điểm của MVC:

- Tách ra từng phần riêng biệt nên có thể thay đổi từng phần độc lập mà không sợ ảnh hưởng đến các thành phần khác (không cần biên dịch lại tất cả vì mỗi thành phần có lớp riêng tương ứng với nó): ví dụ thay đổi giao diện...
- Có thể phân công việc cho nhiều người cùng làm.

II. Thiết kế Đối tượng

- Đối tượng
- Phân loại đối tượng
- Các bước thiết kế đối tượng

II.1 Đối tượng

■ **Đối tượng = Dữ liệu + Xử lý**

Dữ liệu $\Leftrightarrow$ Thuộc tính

Xử lý $\Leftrightarrow$ Phương thức

■ **Đối tượng là thể hiện cụ thể của lớp đối tượng.**

II.2 Phân loại đối tượng

- **Đối tượng nghiệp vụ: *business/domain object***

Dữ liệu + *xử lý nghiệp vụ*

- **Đối tượng hệ thống: *system object***

Dữ liệu + *xử lý hỗ trợ*

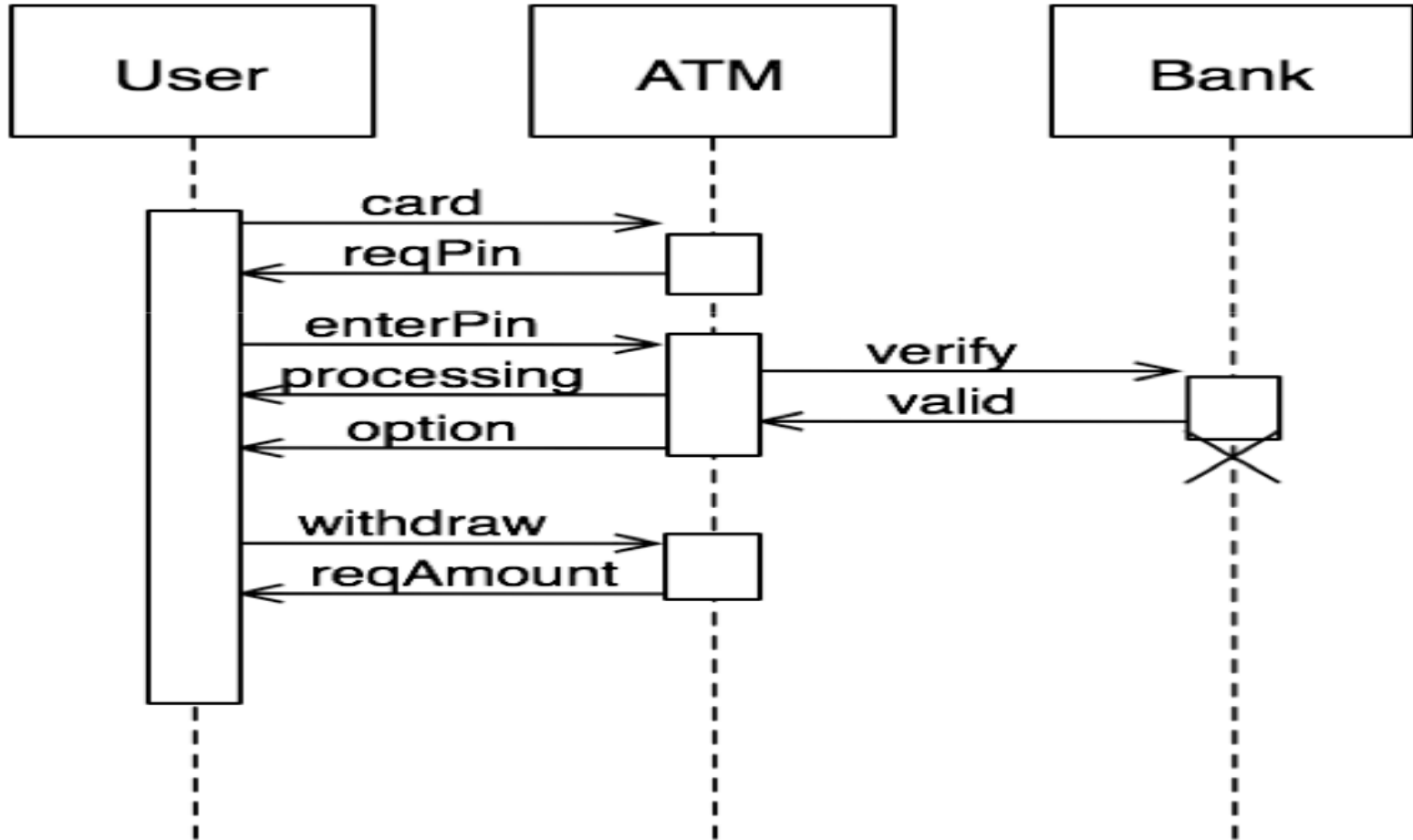
- **Đối tượng giao diện người dùng: *UI object***

Dữ liệu + *xử lý tương tác người dùng*

II.3 Các bước thiết kế đối tượng

- Xây dựng kịch bản tương tác ban đầu.
- Xác định đối tượng **dựa trên các tương tác** của kịch bản ban đầu.
- Chi tiết hoá kịch bản ban đầu **có sự tham gia của các đối tượng đã xác định:**
 - *Phân tích các tương tác thành các xử lý.*
 - *Đối tượng nhận xử lý sẽ chứa các phương thức tương ứng với các xử lý này.*

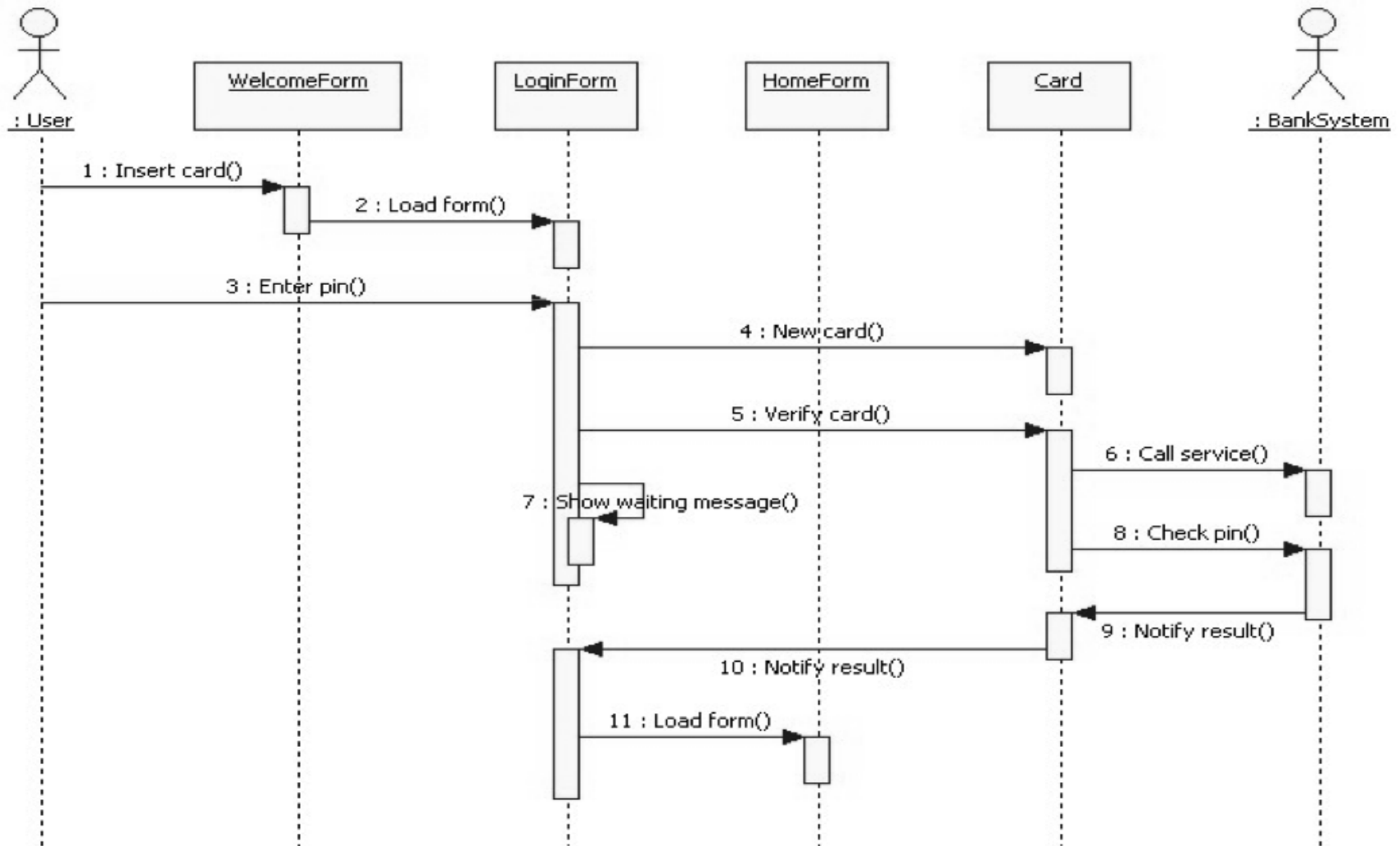
Ví dụ



Ví dụ (tt)

Tương tác	Đối tượng	Loại
card	WelcomeForm	UI
reqPin	LoginForm	UI
enterPin	LoginForm	UI
verify	Card	Domain
processing	LoginForm	UI
valid	Card	Domain
option	HomeForm	UI
withdraw	HomeForm	UI
reqAmount	WithdrawForm	UI

Ví dụ (tt)



Q & A

Câu hỏi ôn tập

- 1) Trình bày khái niệm kiến trúc hệ thống.
- 2) Hãy nêu các bước thiết kế kiến trúc hệ thống.
- 3) Trình bày các kiến trúc hệ thống và cho ví dụ minh họa.
- 4) Hãy nêu các loại đối tượng trong thiết kế đối tượng.
- 5) Trình bày các bước thiết kế đối tượng.

Các loại kiến trúc phổ biến?

- **Triển khai**

- Máy đơn (tương tác, xử lý, lưu trữ trên 1 máy)
- Máy khách-chủ (Client-server)
- Phân tán (Distributed)

- **Phân tầng, lớp**

- 1 tầng
- 2 tầng
- 3 tầng

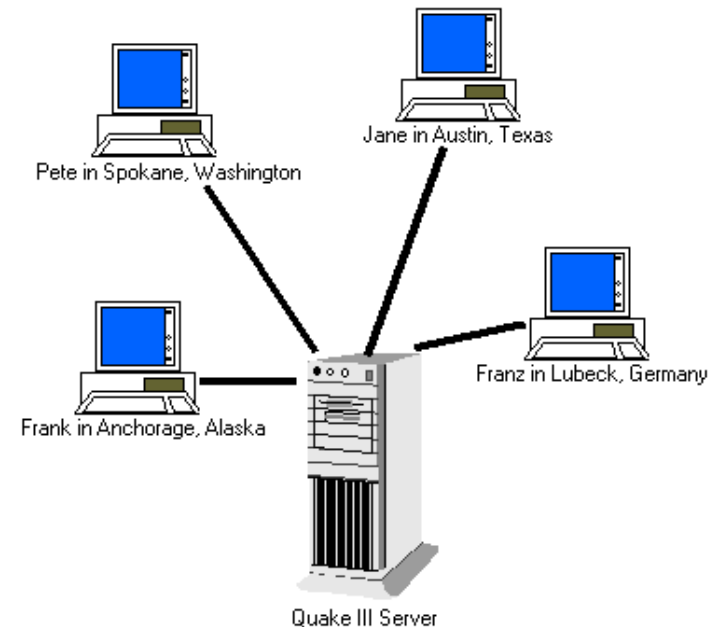
Kiến trúc Client-server?

❖ Client

- ✓ Gửi yêu cầu
- ✓ Đợi phản hồi từ server.
- ✓ Tương tác trực tiếp với end-users thông qua giao diện

❖ Server

- ✓ Đợi, Nhận yêu cầu từ Client
- ✓ Xử lý yêu cầu và phản hồi cho client
- ✓ Thông thường chấp nhận 1 số lượng lớn các kết nối từ các client.
- ✓ Không tương tác trực tiếp với end-user

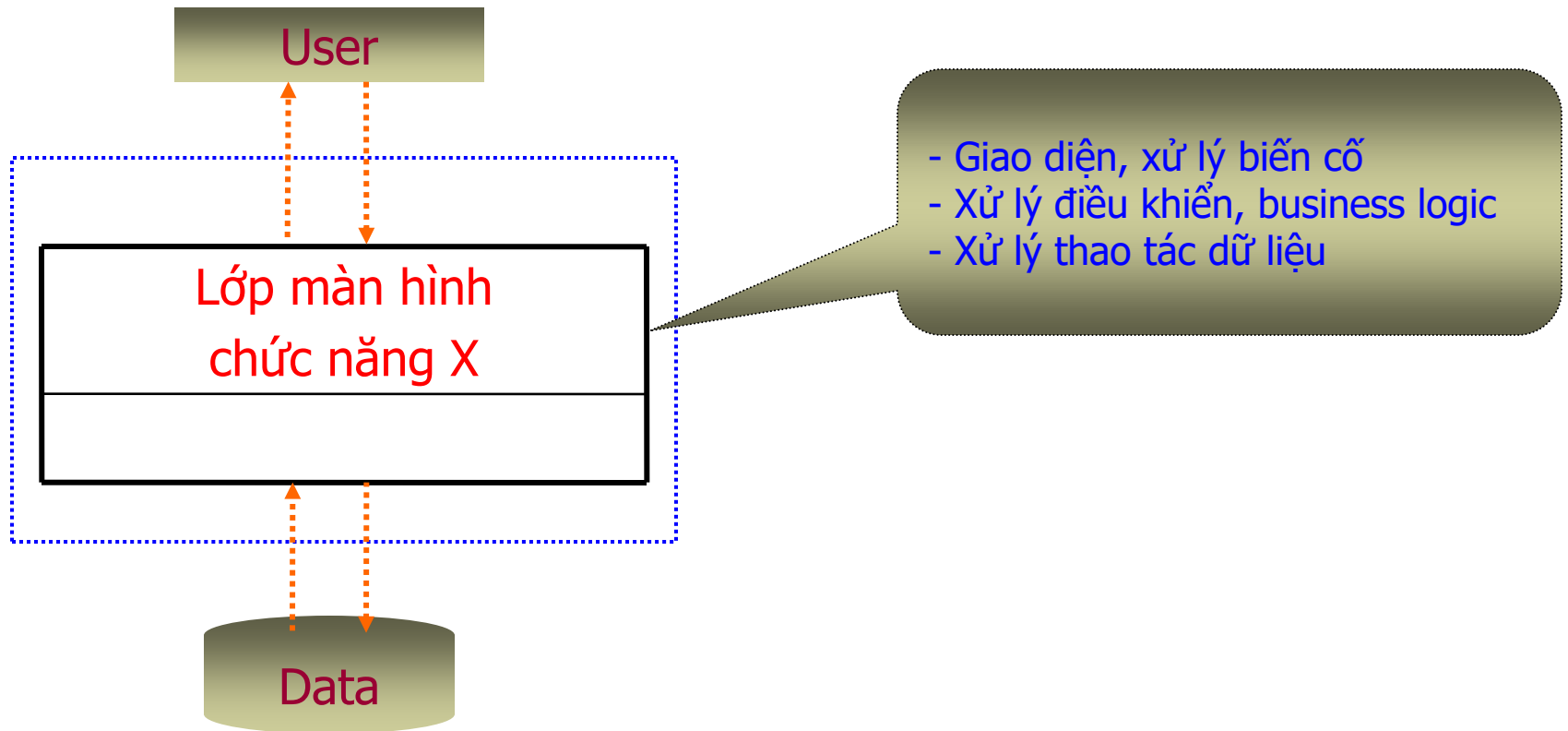


Kiến trúc phân tán (Distributed)?

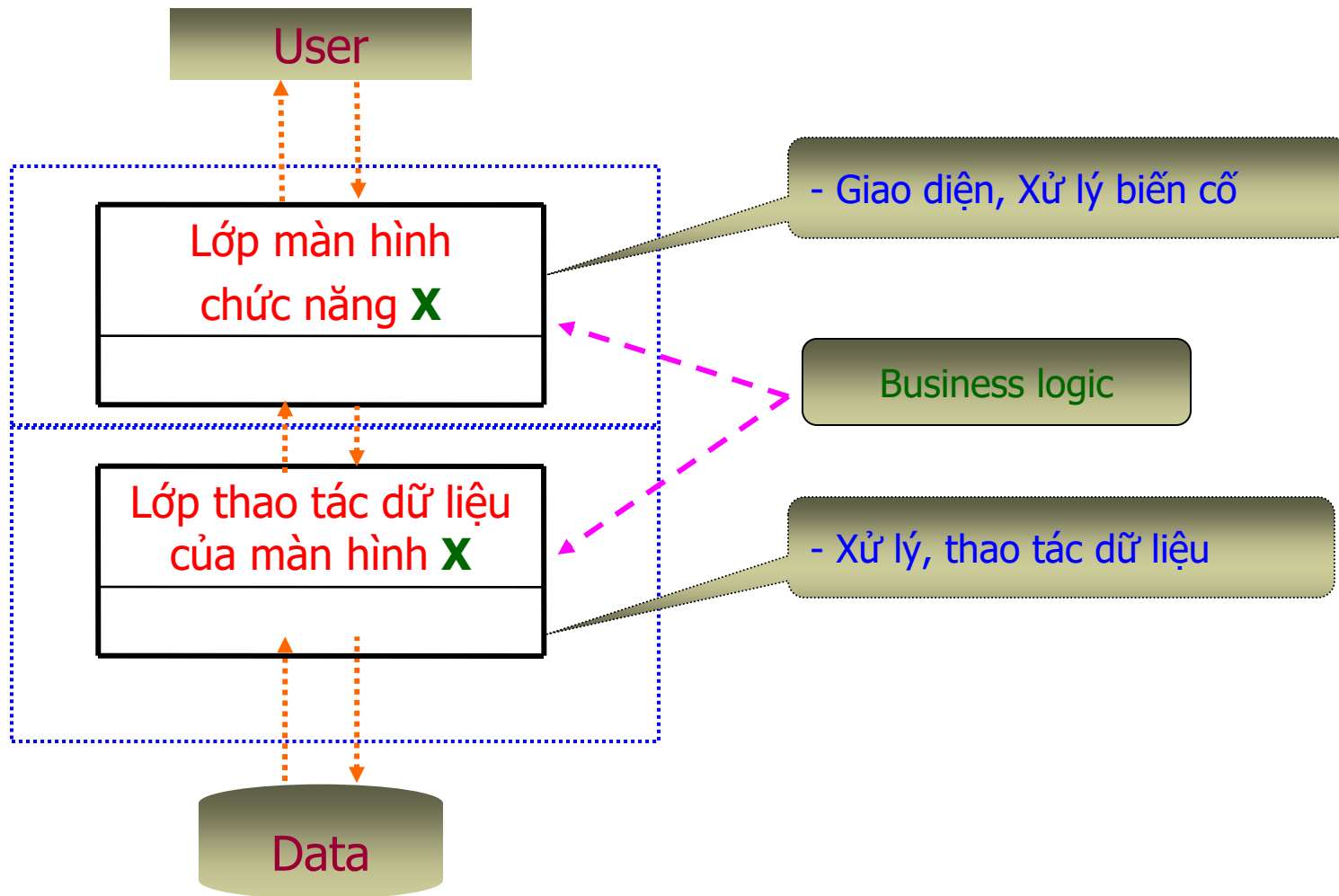
- ❖Chương trình chia thành nhiều phần khác nhau. Mỗi phần có thể xử lý trên các máy khác nhau.



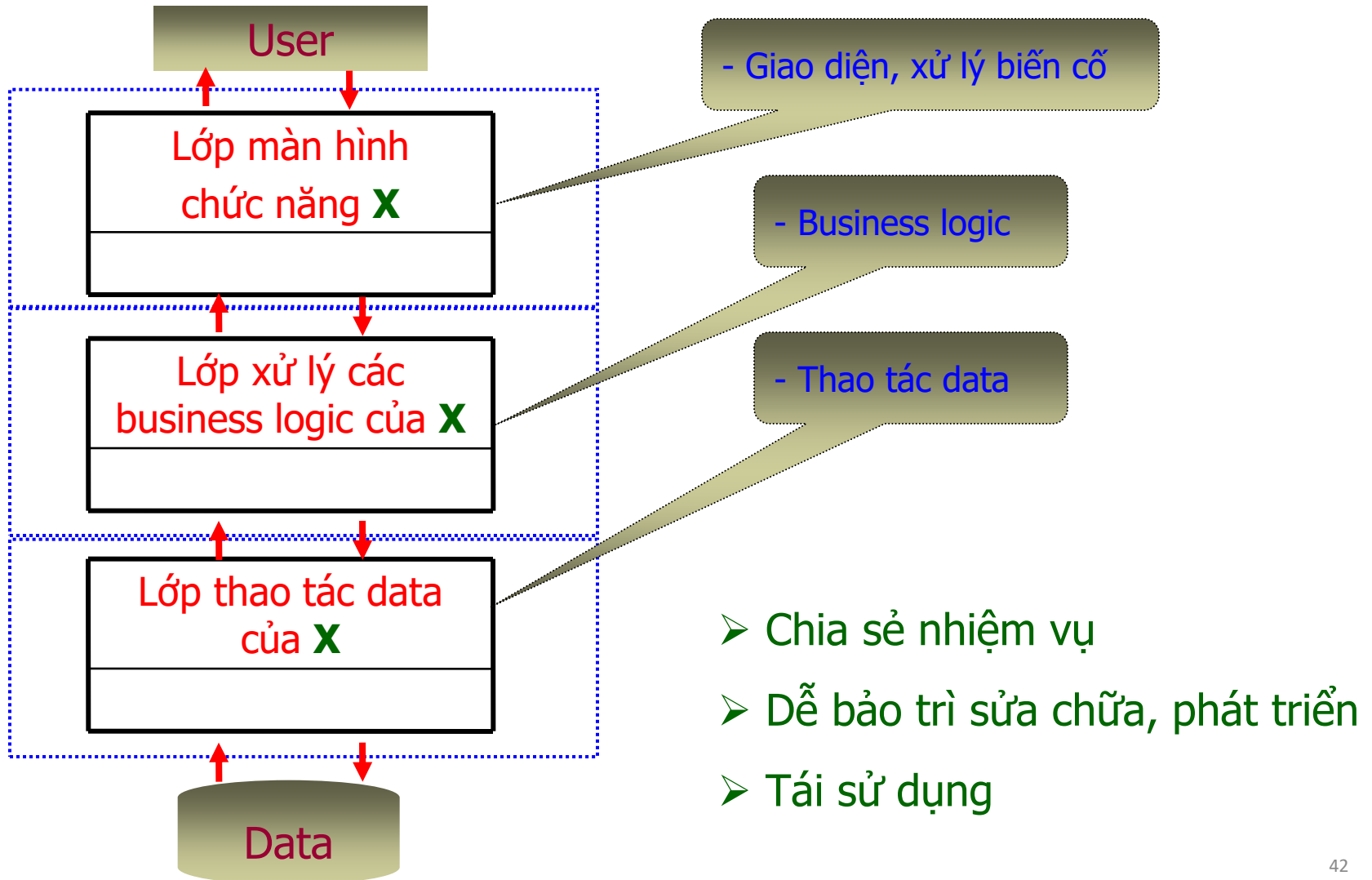
Kiến trúc phân lớp – 1 layer



Kiến trúc phân lớp – 2 layer



Kiến trúc phân lớp – 3 layer



Tiers & Layers

Physical view (Tiers)



Logical view (Layers)

GUI



Business Logic



Data access

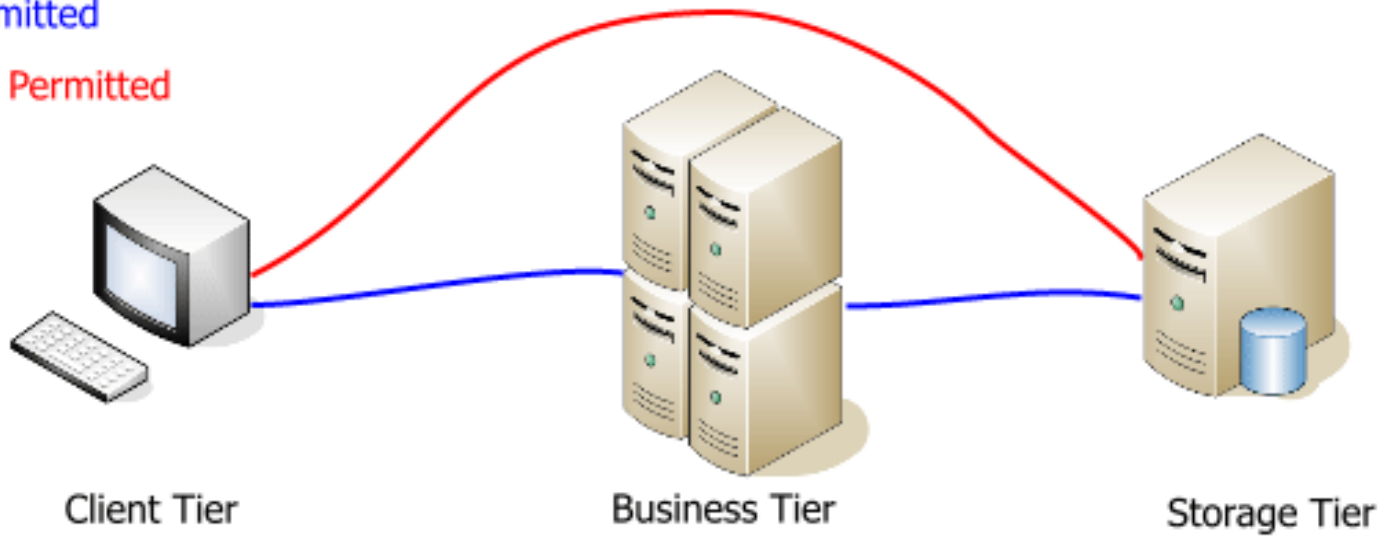


Database

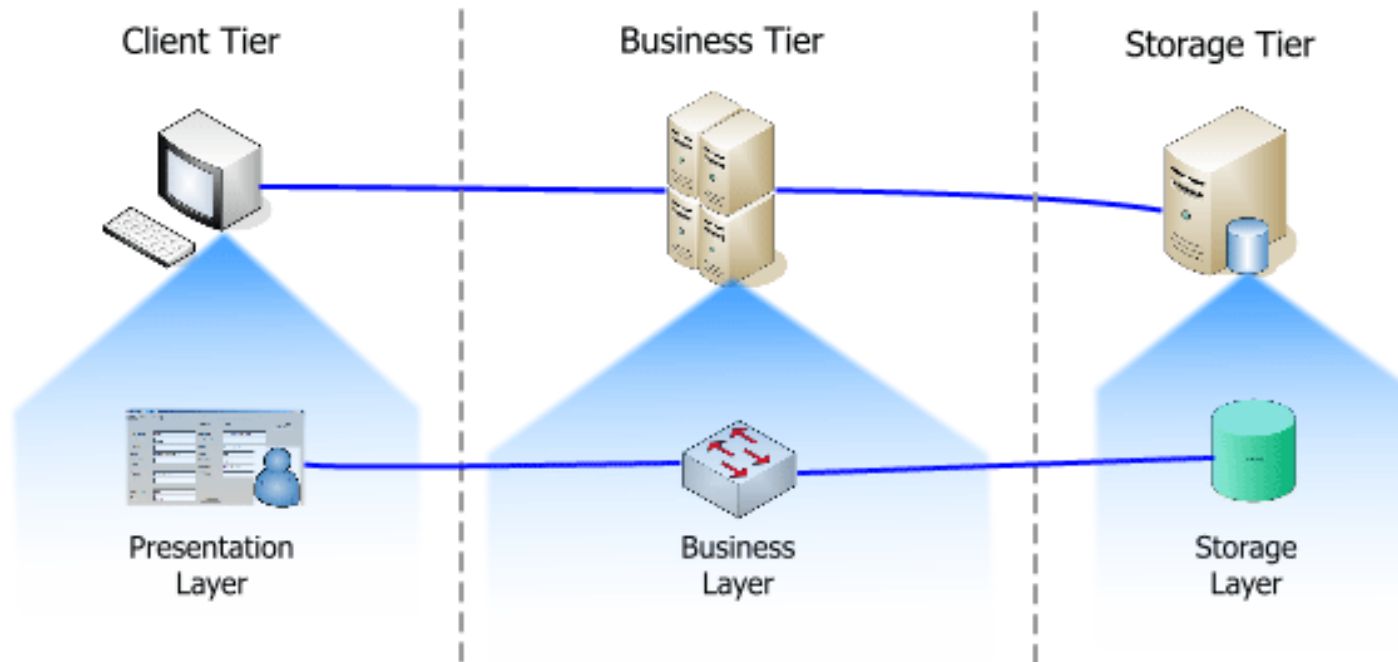
Tiers & Layers

Permitted

Not Permitted

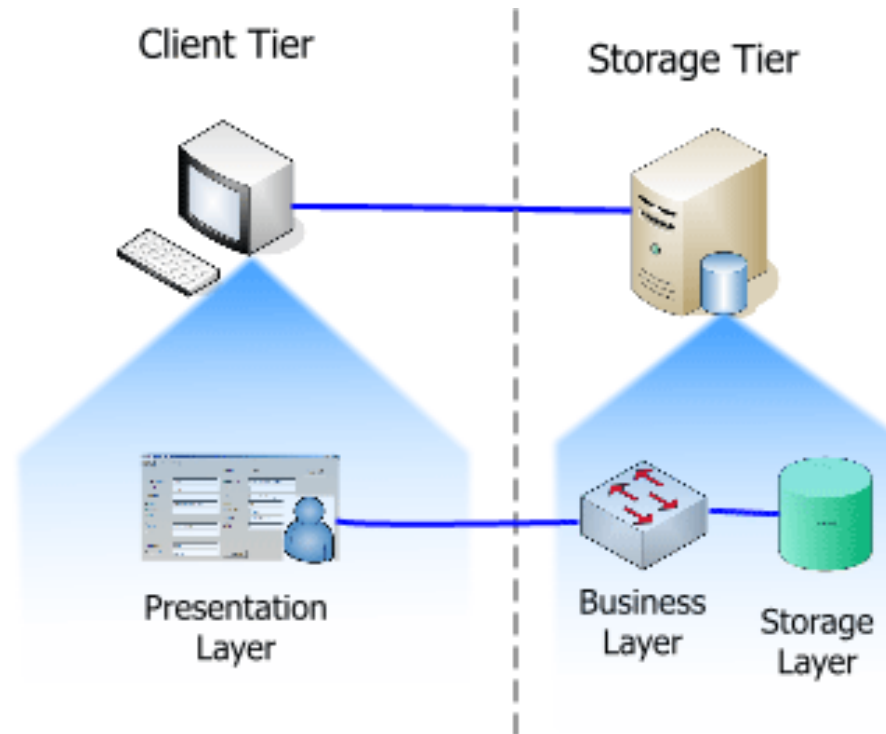


Tiers & Layers



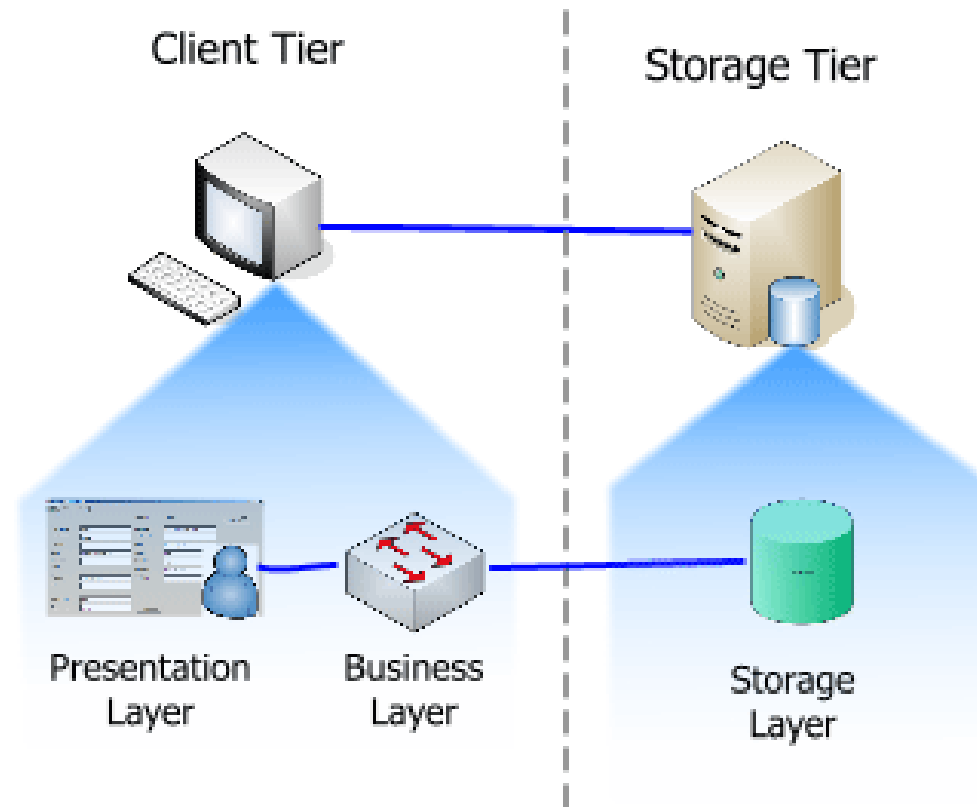
3 tiers & 3 Layers

Tiers & Layers



2 tiers & 3 Layers (Business layer is on Storage tier)

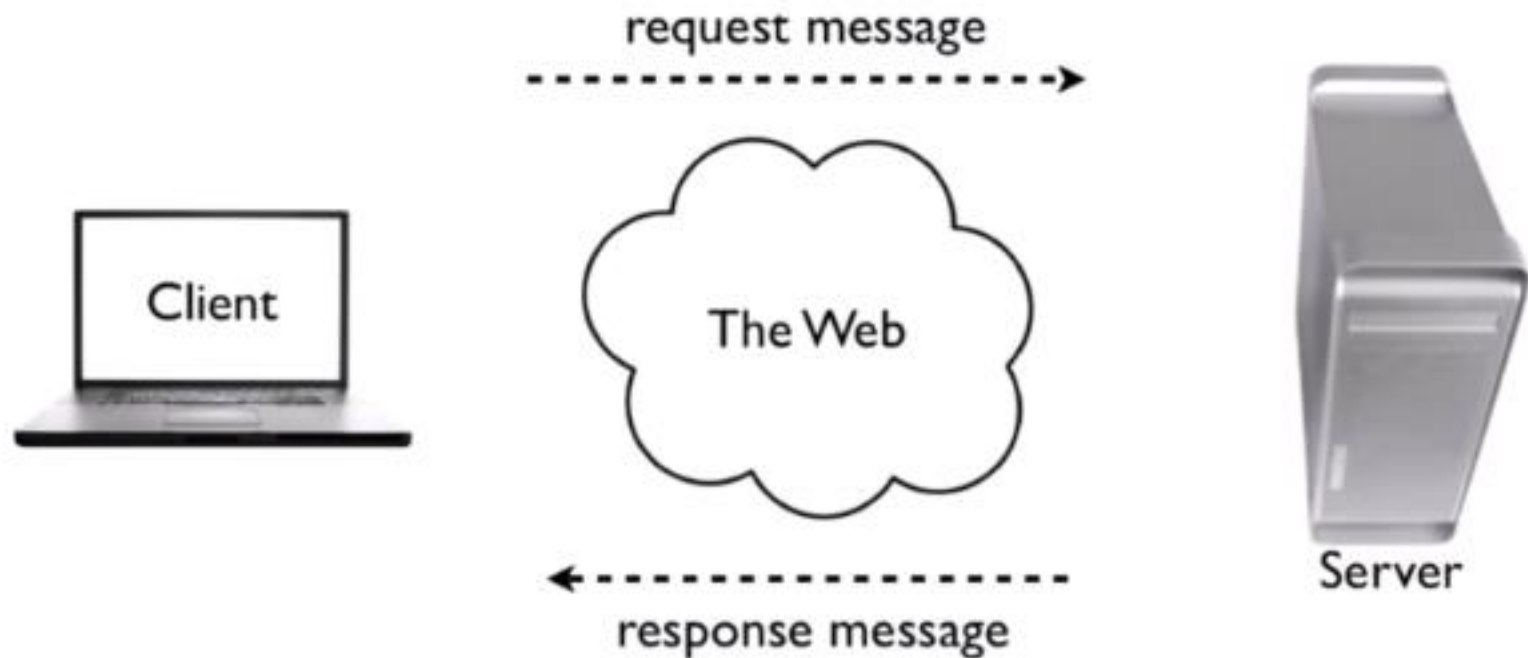
Tiers & Layers



2 tiers & 3 Layers (Business layer is on Client tier)

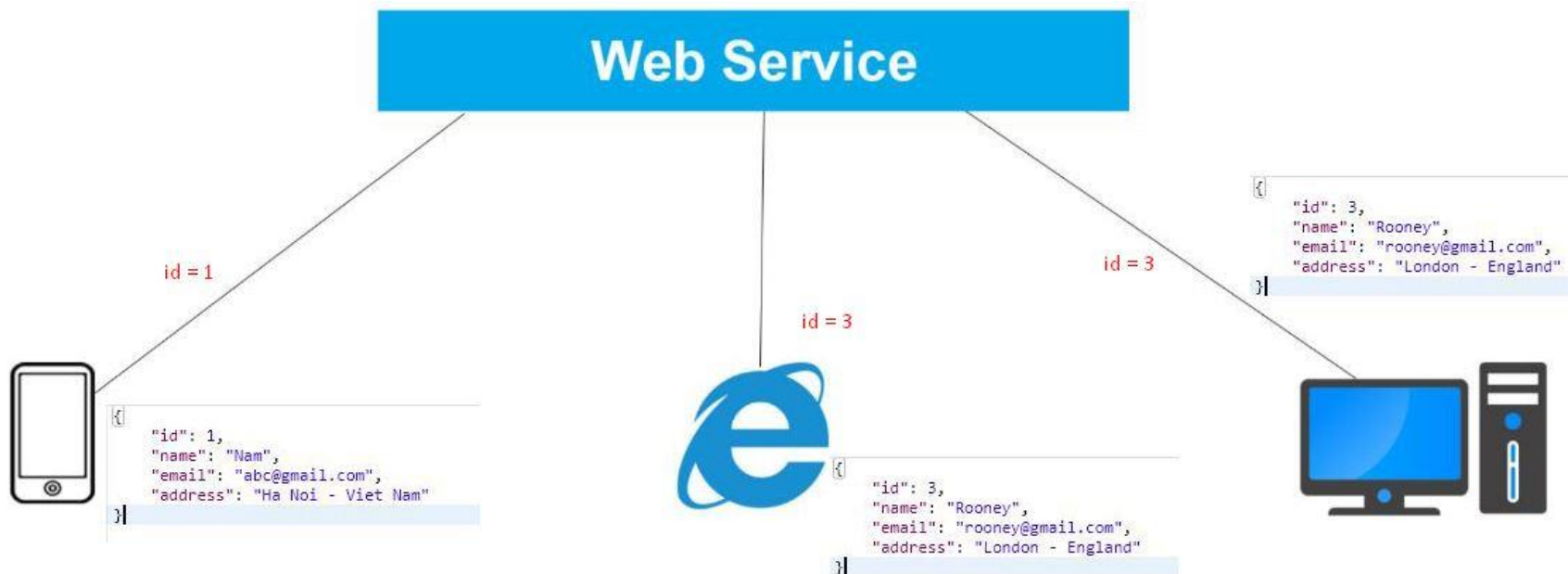
Web Service

Giúp các **ứng dụng có thể tương tác** với nhau giữa những chiếc máy tính **thông qua mạng Internet.**



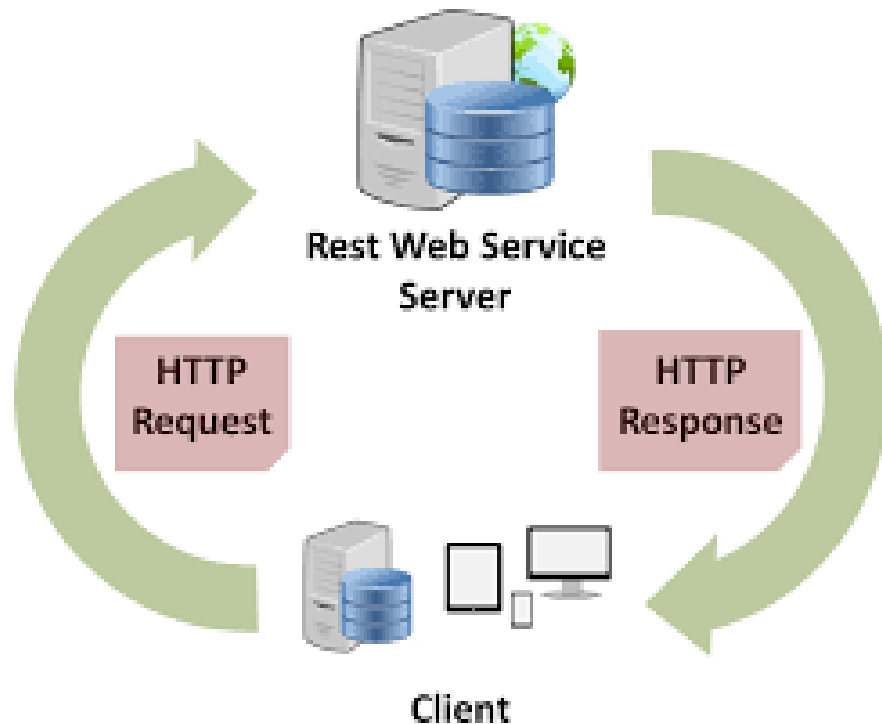
Đặc điểm Web Service

Giúp cho client và server tương tác được với nhau ngay cả khi môi trường của chúng khác nhau.

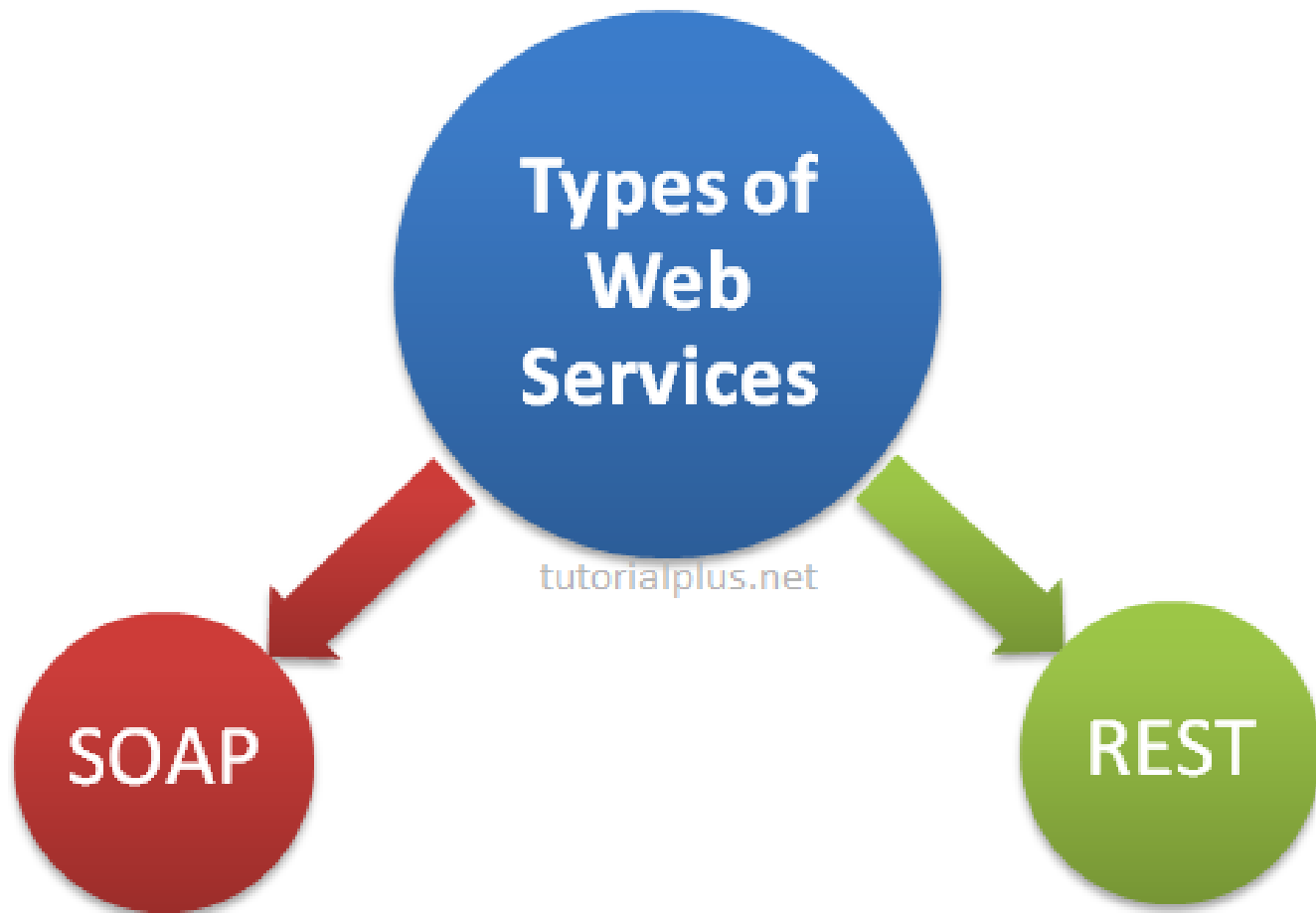


Đặc điểm Web Service

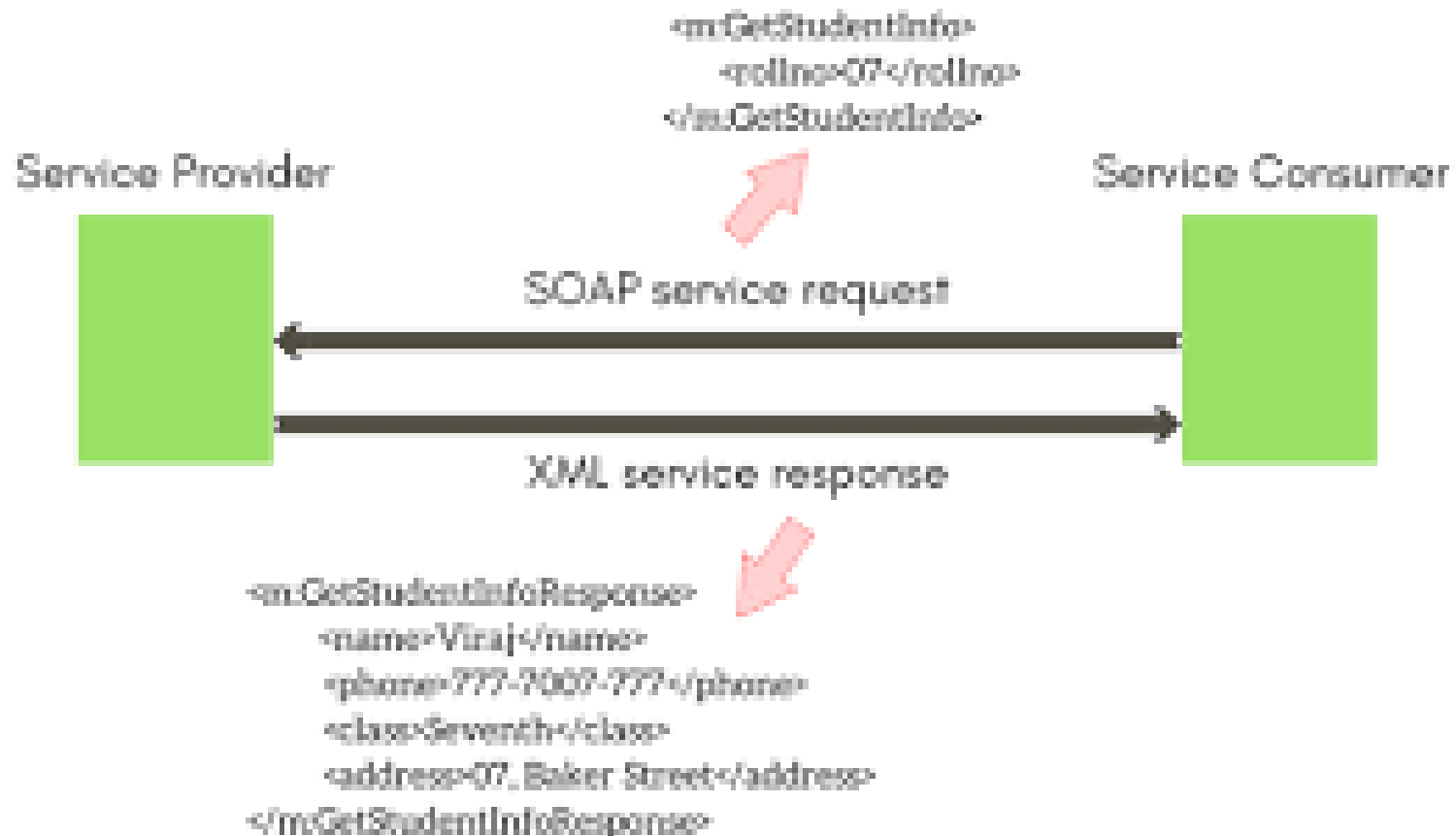
Dễ dàng nâng cấp và tái sử dụng - Web Service chỉ bao gồm các hàm nên việc thay đổi dễ dàng



Web Service – Phân loại



SOAP – Simple Object Access Protocol

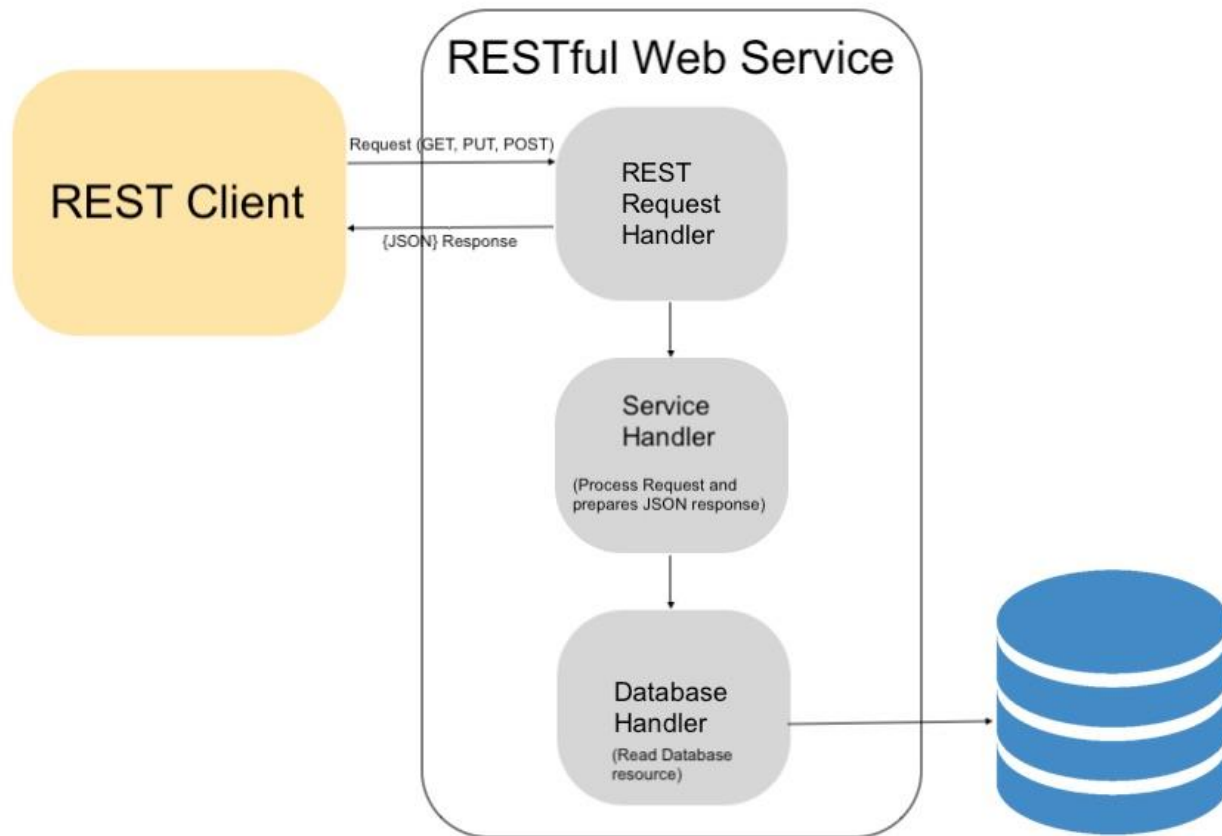


SOAP — Simple Object Access Protocol

- SOAP a XML-based protocol for accessing web services
- SOAP is platform and language independent
- SOAP is a W3C recommendation for communication between two applications

RESTful— Representational State Transfer

REST is an architectural style not a protocol



SOAP VS RESTFULL

Type	Advantages	Description
SOAP	Security	SOAP defines its own security known as WS Security
	Language and Platform independent	SOAP web services can be written in any programming language and executed in any platform
RESTful	Fast	RESTful Web Services are fast
	Language and Platform independent	It is also language and platform independent
	Can use SOAP	RESTful web services can use SOAP web services as the implementation
	Permits different data format	RESTful web service permits different data format such as Plain Text, HTML, XML and JSON

Đặc điểm RESTful Web Service

- Sử dụng các **phương thức HTTP/HTTPs**
- **Phi trạng thái (stateless)** (không lưu trữ thông tin của client)
- Hiện thị **cấu trúc thư mục như các URIs**
- Truyền tải dữ liệu với **định dạng phong phú**: html, json, text, xml...

HTTP Method RESTful

RESTful Web Service

PUT

DELETE

GET

POST

HTTP Method RESTful hỗ trợ

- **GET**: được sử dụng để truy vấn, lấy dữ liệu từ server. Request sử dụng method GET chỉ nên sử dụng để lấy dữ liệu, không nên chỉnh sửa/tác động tới dữ liệu
- **POST**: được sử dụng để gửi dữ liệu tới server, tạo mới tài nguyên, ví dụ upload file, đăng nhập
- **HEAD**: giống như method GET nhưng chỉ nhận về status và header, không nhận về body

HTTP Method RESTful hỗ trợ

- **PUT**: được dùng để cập nhật cho tài nguyên (ví dụ update cả đối tượng user)
- **PATCH**: giống như method PUT nhưng chỉ cập nhật cho 1 phần dữ liệu (ví dụ chỉ update thông tin 1 thuộc tính của user)
- **DELETE**: được dùng cho hành động xóa đối tượng trên server.
- Một số method khác: **CONNECT, OPTIONS, TRACE**

JSON

- **JSON** - Javascript **O**bject **N**otation: là một dạng dữ liệu **tuân theo một quy luật** nhất định

```
1  {  
2      "sv0001" : {  
3          "toan" : "Môn Toán",  
4          "ly" : "Môn Lý"  
5      },  
6      "sv0002" : {  
7          "toan" : "Môn Toán",  
8          "anh" : "Môn Anh"  
9      }  
10 }
```

Định dạng dữ liệu JSON

- Đặt tên biến ta thường đặt kiểu camel case ví dụ dateOfBirth hoặc kiểu snake case: date_of_birth

```
// snake_case
{
  date_of_birth: "2017/12/01"
}

// camelCase
{
  dateOfBirth: "2017/12/01"
}
```

Định dạng URI

- Để chia tách các từ trong URI ta thường dùng dấu gạch ngang hoặc gạch dưới (áp dụng cho các url cho các trang web)

Ví dụ:

`https://stackjava.com/web-service/restful-web-service.html`

hoặc

`https://stackjava.com/web-service/restful_web_service.html`

Cấu trúc URI

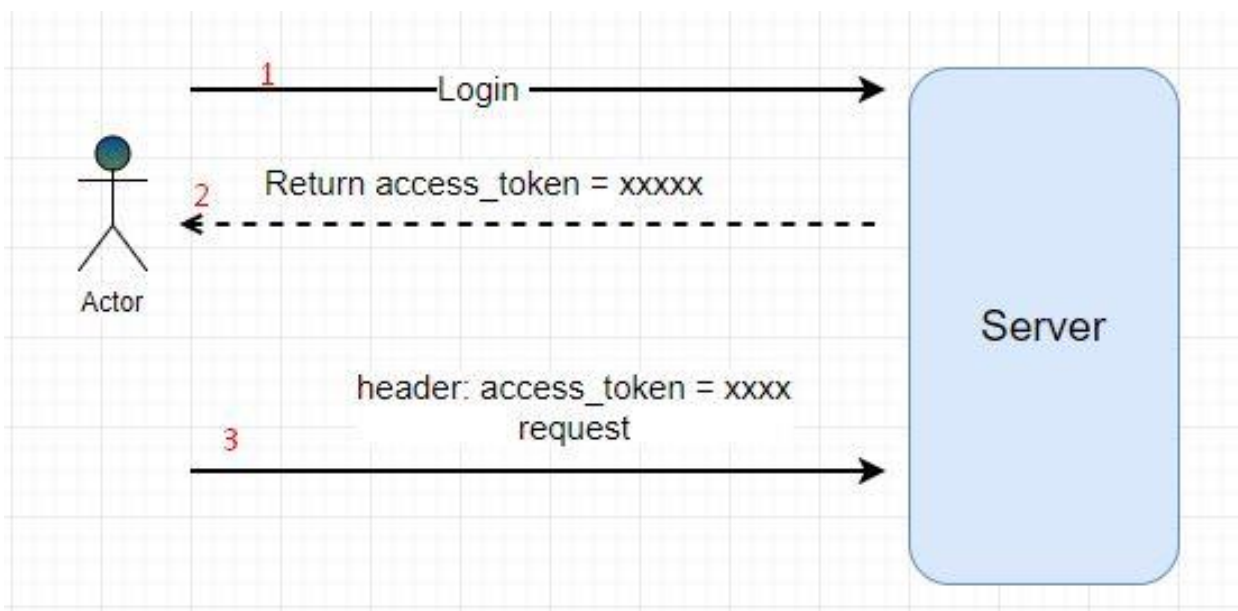
- Việc đặt URI cho các api web service thường đặt theo kiểu cấu trúc thư mục, nhiều hành động chung 1 URI nhưng khác method

Ví dụ:

- **GET** /user: lấy thông tin tất cả user
- **GET** /user/1: lấy thông tin của user có id = 1
- **POST** /user: tạo user mới
- **PUT** /user/1: cập nhật đối tượng user có id = 1
- **PATCH** /user/1: cập nhật 1 thuộc tính cho đối tượng user có id = 1
- **DELETE** /user/1: xóa đối tượng user có id = 1

Bảo mật, xác thực trong restful web service

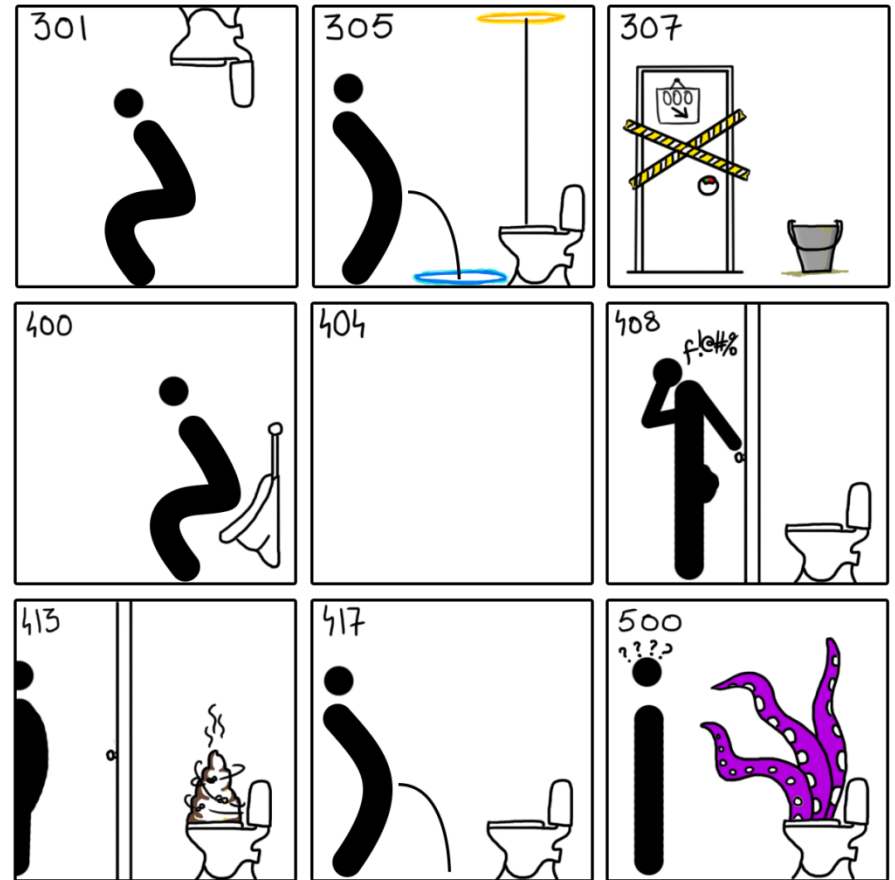
- Request trong web service là stateless, nó cũng không sử dụng cookies nên ta không thể lưu thông tin đăng nhập vào session.
- Để xác thực trong RESTful web service, sau khi đăng nhập client sẽ nhận được 1 token, mỗi lần gửi request sẽ gửi kèm token đó vào header của request. Server sẽ dựa vào token để xác thực client



HTTP STATUS CODE

- HTTP status code (mã trạng thái) là mã code server trả về sau mỗi lần gửi request. Tất cả các request mà server nhận được đều sẽ được trả về 1 response với 1 mã code tương ứng.

HTTP STATUS CODES



MONKEYUSER.COM

Các loại HTTP STATUS CODE

- **1xx** (Informational – Thông tin): Các status code loại này dùng để đơn giản thông báo với client rằng server đã nhận được request. Các status code này ít được sử dụng.
- **2xx** (Success – Thành công): Các status code loại này có ý nghĩa rằng request được server và xử lý thành công.
- **3xx** (Redirect – Chuyển hướng): Các status code loại này có ý nghĩa rằng server sẽ chuyển tiếp request hiện tại sang một request khác và client cần thực hiện việc gửi request tiếp theo đó để có thể hoàn tất. Thông thường khi trình duyệt nhận được status code loại này nó sẽ tự động thực hiện việc gửi request tiếp theo để lấy về kết quả.

Các loại HTTP STATUS CODE

- **4xx** (Client error – Lỗi Client): Các status code loại này có ý nghĩa rằng đã có lỗi từ phía *client* trong khi gửi request. Ví dụ như sai URL, sai HTTP method, không có quyền truy cập vào trang...
- **5xx** (Server error – Lỗi Server): Các status code loại này có ý nghĩa rằng đã có lỗi từ phía server trong khi xử lý request. Ví dụ như server quá tải, hết bộ nhớ, lỗi kết nối database...

Một số ví dụ HTTP STATUS CODE

- 200 OK – Trả về thành công cho những phương thức GET, PUT, PATCH hoặc DELETE.
- 201 Created – Trả về khi một Resource vừa được tạo thành công.
- 400 Bad Request – Request không hợp lệ
- 401 Unauthorized – Request cần có sự authentication.
- 404 Not Found – Không tìm thấy resource từ URI
- ...

